

Deep Learning HW2

111511141 賴昱凱

1. Using Convolutional Neural Network for Image Recognition

1-1

design your network architecture and analyze the effect of different stride size and filter size

I used a total of 6 convolutional layers with the ReLU activation function and applied max-pooling after the 1st and 6th convolutional layers to reduce the image size from 28x28 to 7x7, halving the size each time. I also added residual links between conv3 and conv4, and between conv5 and conv6. This helps to avoid the problem of gradient vanishing. After these layers, the image passes through 3 linear layers, and the final output consists of 10 neurons representing the digits 0 to 9.

```
class Net(nn.Module):
    def __init__(self, num_classes=10):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=5, stride=1, padding=2)
        self.maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=5, stride=1, padding=2)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2)
        self.conv4 = nn.Conv2d(64, 64, kernel_size=5, stride=1, padding=2)
        self.conv5 = nn.Conv2d(64, 64, kernel_size=5, stride=1, padding=2)
        self.conv6 = nn.Conv2d(64, 64, kernel_size=5, stride=1, padding=2)
        self.fc1 = nn.Linear(64 * 7 * 7, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        conv1_out = x
        x = self.maxpool(x)
        x = F.relu(self.conv2(x))
        conv2_out = x
        x = F.relu(self.conv3(x))
        conv3_out = x
        x = F.relu(self.conv4(x) + x)
        conv4_out = x
        x = F.relu(self.conv5(x))
        conv5_out = x
        x = F.relu(self.conv6(x) + x)
        conv6_out = x
        x = self.maxpool(x)
        x = torch.flatten(x, start_dim=1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x, [conv1_out, conv2_out, conv3_out, conv4_out, conv5_out, conv6_out]
```

Analyze the effect of different stride size and filter size :

1. Definition

Stride :

Stride is the step size by which the convolutional filter moves across the input image. Therefore, increasing the stride value will cause the output image to become smaller if other parameters, such as padding, are not modified.

Filter size :

Also known as kernel size, it refers to the dimensions of the convolutional filter, typically represented as a $k \times k$ matrix. The convolutional filter slides over the input image and applies the convolution operation.

2. Effect

Stride and filter size affect several things of the CNN, including the size of the output feature map, the amount of computation, and the ability to capture features.

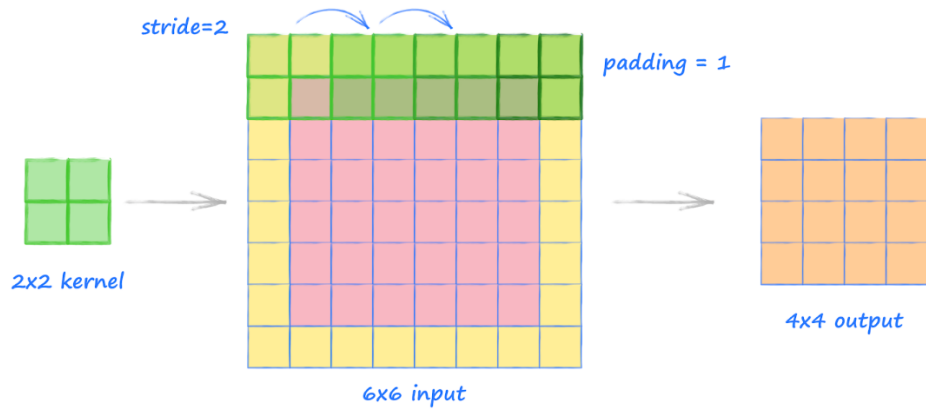
For the size of the output feature map, here's the formula :

W, H : size of input feature map, F : filter(kernel) size, P : padding, S : stride

$$\text{output width} = \frac{W - F_w + 2P}{S_w} + 1$$

$$\text{output height} = \frac{H - F_h + 2P}{S_h} + 1$$

Basically, we can simply prove it by drawing the process of convolution.



For the computation cost, increasing the stride or decreasing the filter size will reduce the computation cost. That's because we reduce the number of matrix multiplications by taking larger steps or decreasing the matrix size. The formula of matrix multiplication is down below.

$$D_k \times D_k \times M \times (N \times D_f \times D_f)$$

Since convolution is computed by taking the region defined by the kernel size, multiplying it element-wise, and summing the results to obtain one pixel of the output. Therefore the computation required for a single output pixel is $D_k \times D_k \times M$, where $D_k \times D_k$ is the total number of pixels in the kernel and M is the number of input channels. If we want the output to have a size of $D_f \times D_f$ and N output channels, the total computation required would be $D_k \times D_k \times M \times (N \times D_f \times D_f)$.

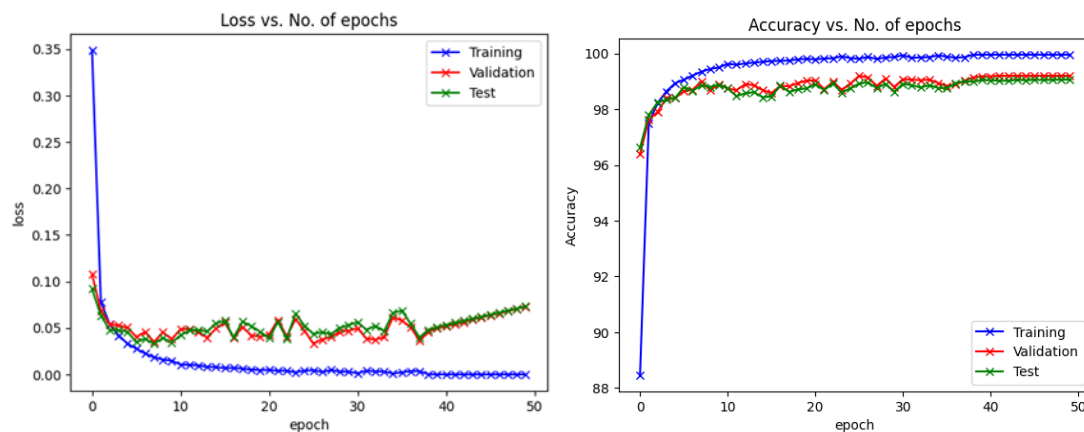
For the ability to capture features of the image, it mostly depends on the filter size (if other parameters are within a normal range). A small filter size allows the model to focus on small but essential features, such as the color of the eyes for recognizing the species of people. On the other hand, a larger filter size causes the model to focus on larger features, such as an animal's color and shape. We can choose the filter size according to the purpose of our models.

For my model, the stride = 1 and the filter size = 5

Since the input image is 28*28 pixels, I think a filter size of 5 perfectly matches the size of number's features, such as the hole in 6 and 8, the curve in 3, and the angle in 4. To avoid losing any features in the image, I decided to use a stride of 1.

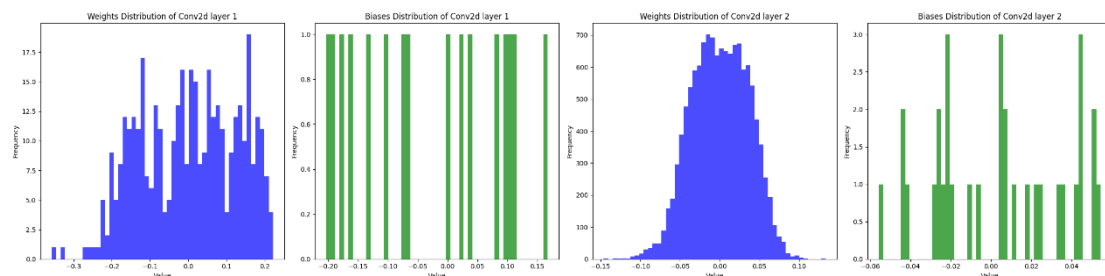
To ensure that the output feature maps have the same size as the input feature maps, I used a padding of 2. This padding ensures that the size of the output feature maps does not decrease continuously, according to the formula mentioned above.

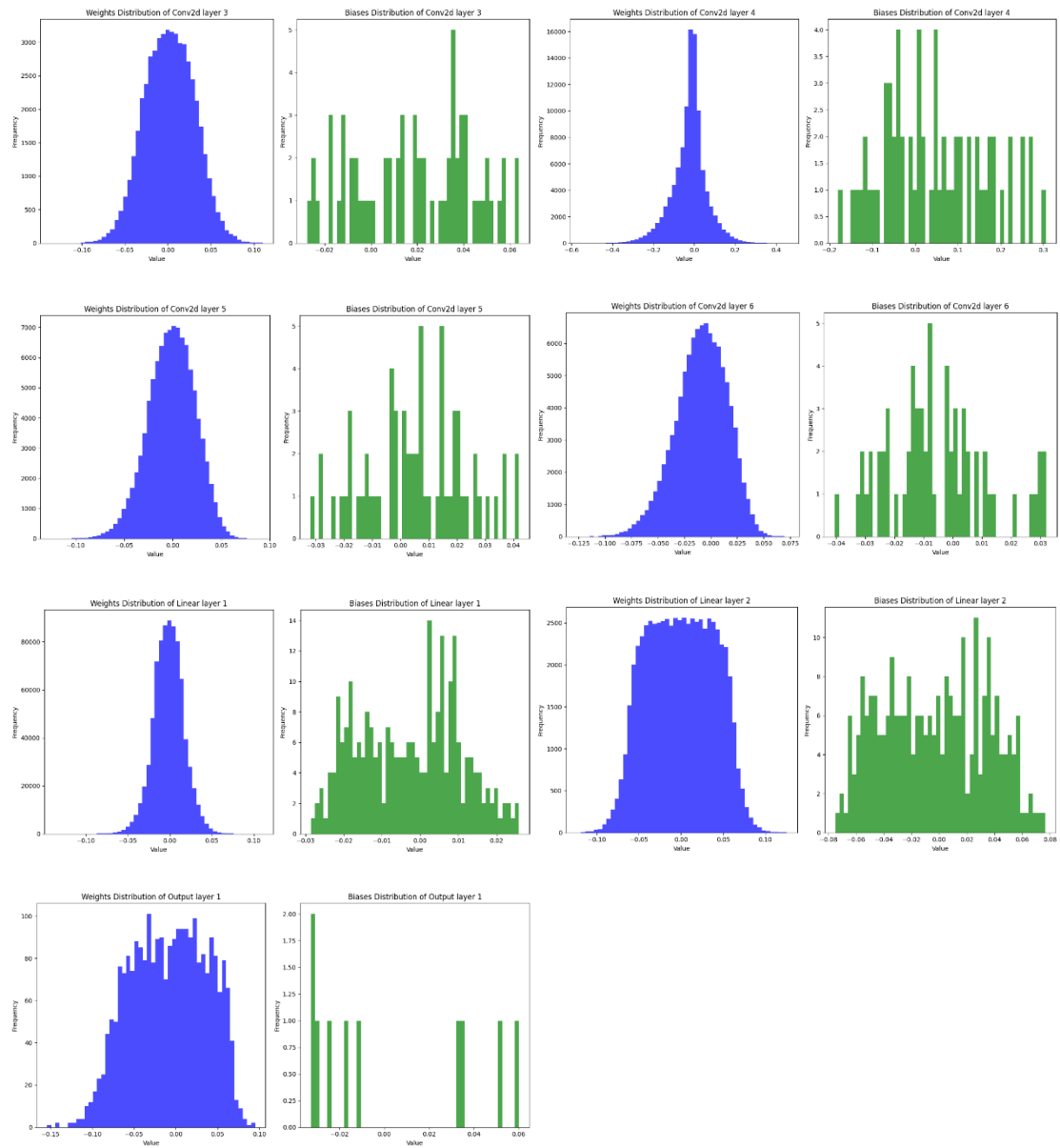
learning curve and accuracy of training, validation and test sets



distributions of weights (blue) and biases (green)

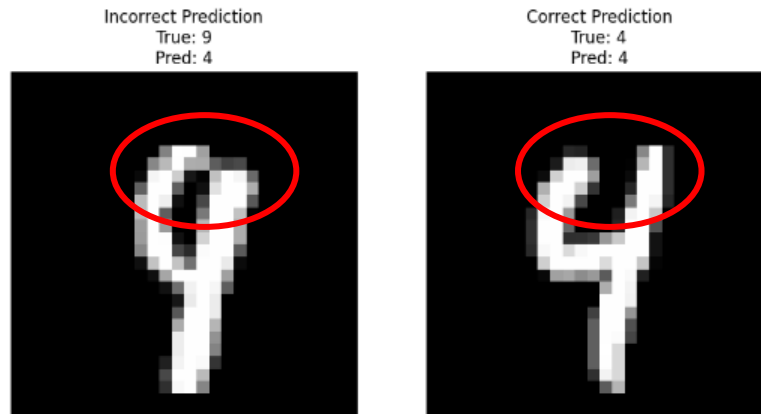
for 6 convolution layers and 3 linear layers





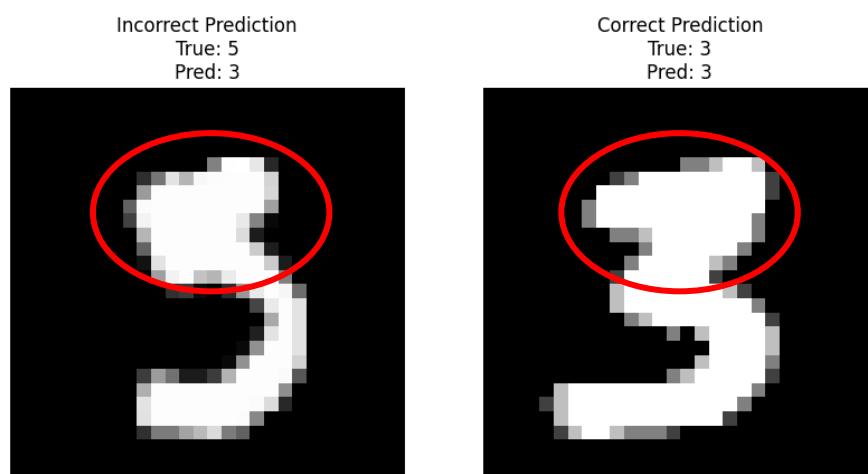
1-2

Show some examples of correctly classified and miss-classified images and discuss your results.



The example I chose is the number 4 and 9. On the left-hand side, it shows the model recognizing it as a 4 instead of a 9, and on the right-hand side is an image correctly recognized as a 4.

The biggest difference between the two images is the bottom of the number, the 9 is closed while the 4 is open. However, the 9 is not perfectly closed, so the model may interpret it as an open shape, resulting in it being recognized as a 4.



Another example I chose is the numbers 3 and 5. On the left-hand side, it shows the model recognizing it as a 3 instead of a 5, and on the right-hand side is an image correctly recognized as a 3.

The biggest difference between the two numbers is the side of the upper connection, the 5 is connected on the left side while the 3 is on the right. However, the upper region of the 5 is messy, so the model can't correctly recognize the side of the connection, resulting in it being recognized as a 3

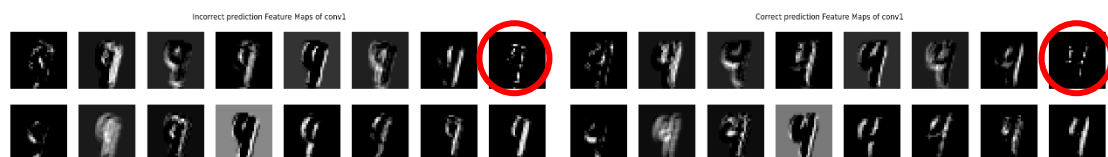
1-3

Following 1-2, observe the feature maps from different convolutional layers and describe how a feature map changes with increasing depth.

Here are the feature maps output from each convolutional layer. Since the output channels of each layer are 16, 32, 64, 64, 64, and 64, the number of images for each layer's outputs will match the number of its output channels.

Left : incorrect prediction / Right : correct prediction

Convolution layer 1 (output channel : 16)



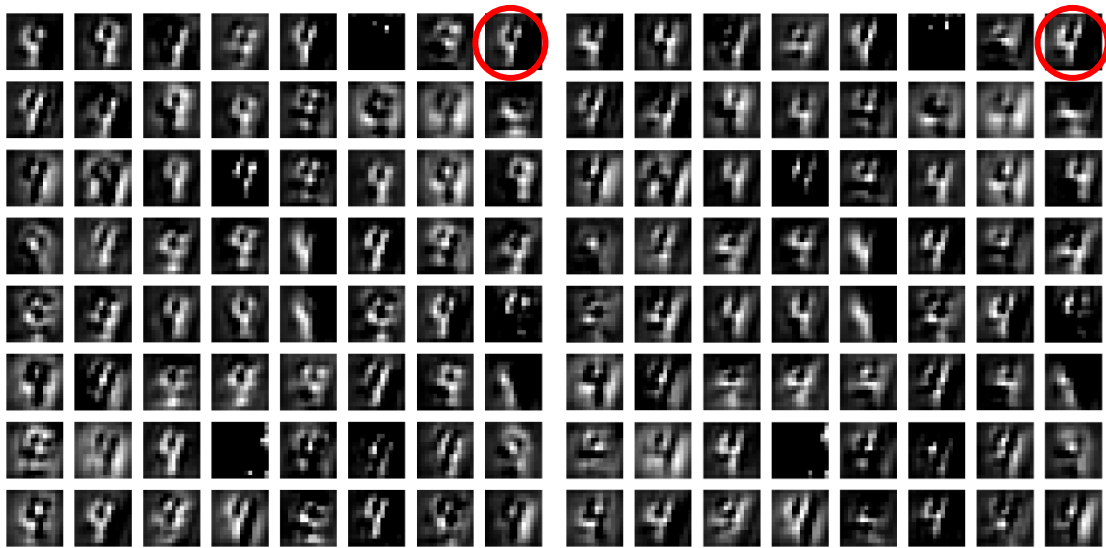
Convolution layer 2 (output channel : 32)



Convolution layer 3 (output channel : 64)

Incorrect prediction Feature Maps of conv3

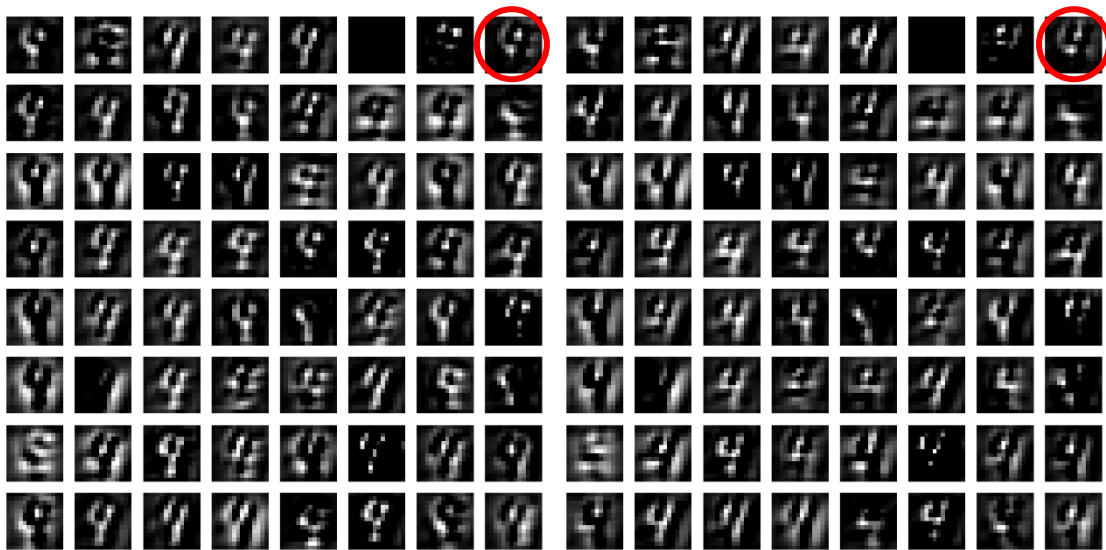
Correct prediction Feature Maps of conv3



Convolution layer 4 (output channel : 64)

Incorrect prediction Feature Maps of conv4

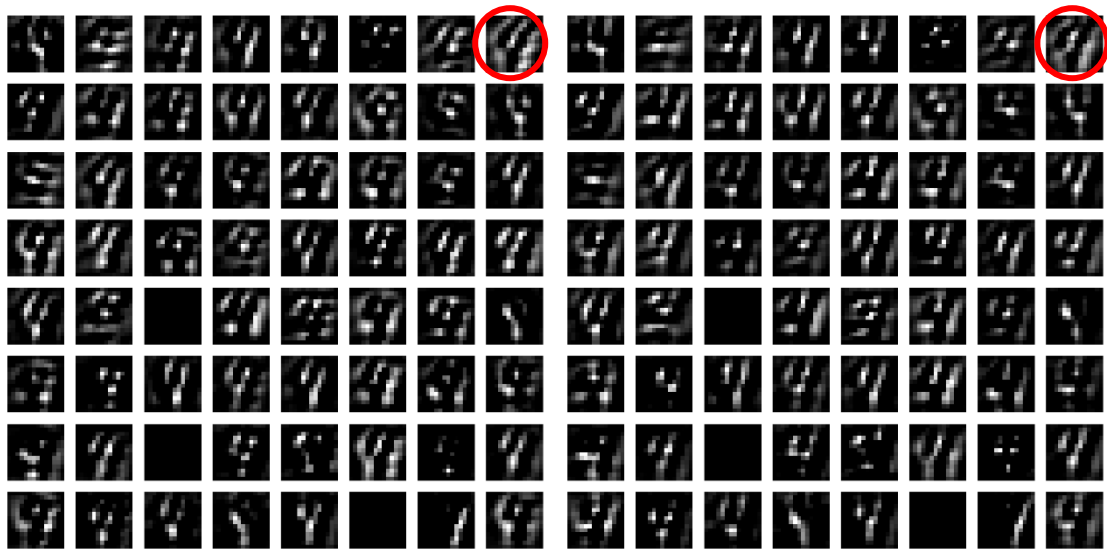
Correct prediction Feature Maps of conv4



Convolution layer 5 (output channel : 64)

Incorrect prediction Feature Maps of conv5

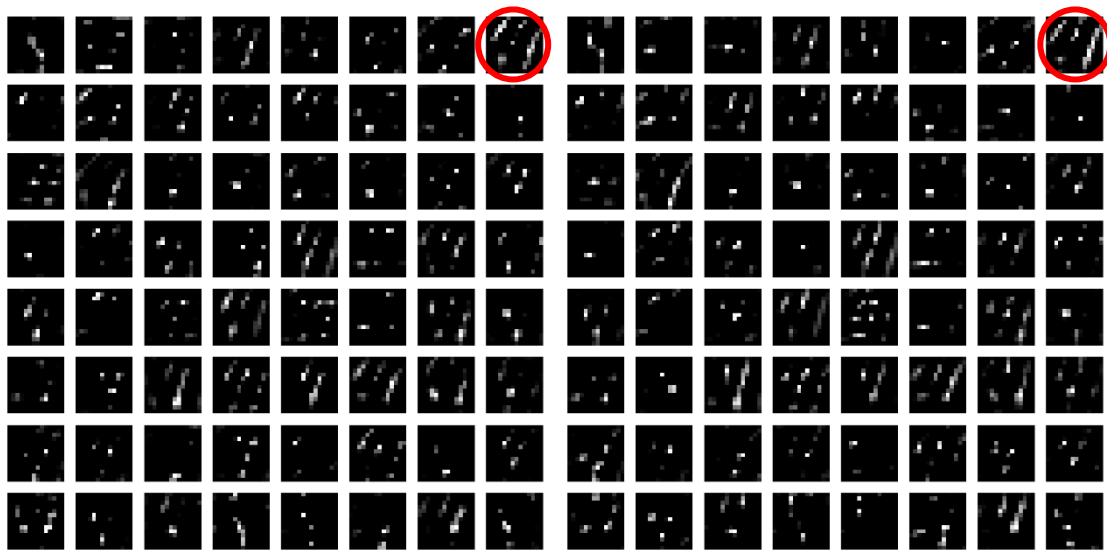
Correct prediction Feature Maps of conv5



Convolution layer 6 (output channel : 64)

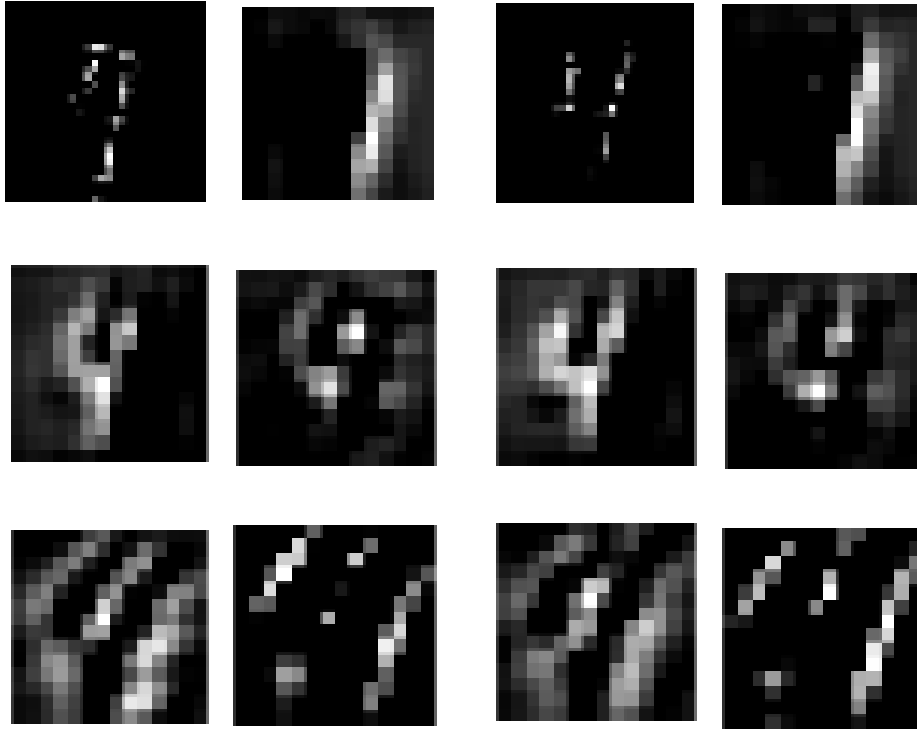
Incorrect prediction Feature Maps of conv6

Correct prediction Feature Maps of conv6



Select a random feature map (red circle) from each convolution layer
and compare the two images

Left : incorrect prediction ($9 \rightarrow 4$) Right : correct prediction ($4 \rightarrow 4$)



As we can see, the feature maps of the two digits have a high similarity. As the input image goes deeper into the network, the shape of the number becomes vaguer. However, the deeper the layer, the clearer the features of the digit become, regardless of where or how the digit is written.

1-4

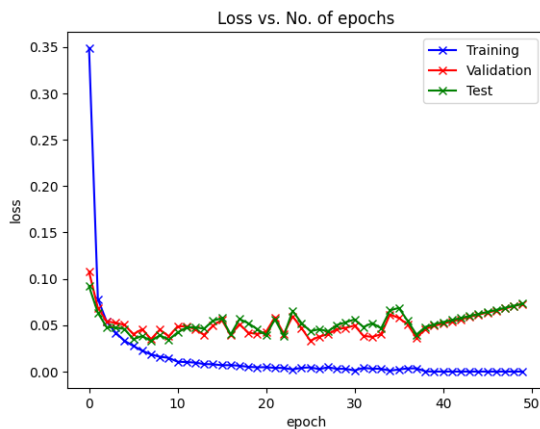
Following 1-1, please add L2 regularization to the CNN implemented in 1-1 and discuss its effect.

```
# L2 regularization
optimizer = torch.optim.Adam(model.parameters(), lr=config['learning_rate'], weight_decay=1e-3)
```

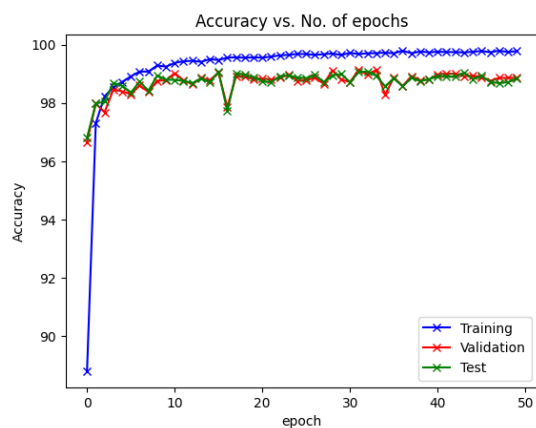
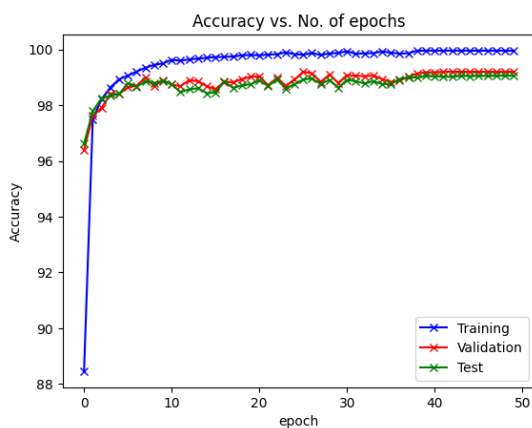
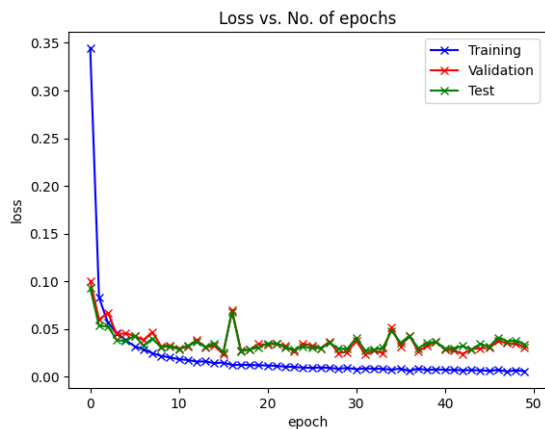
L2 regularization, also known as weight decay, is a technique used to prevent overfitting in models. It works by adding a penalty term to the loss function that is proportional to the sum of the squared values of the weights. This encourages the network to keep the weights small, which can help improve generalization. Generalization refers to the model's ability to perform well on unseen data, ensuring that it maintains good performance.

Loss and Accuracy

No L2 regularization



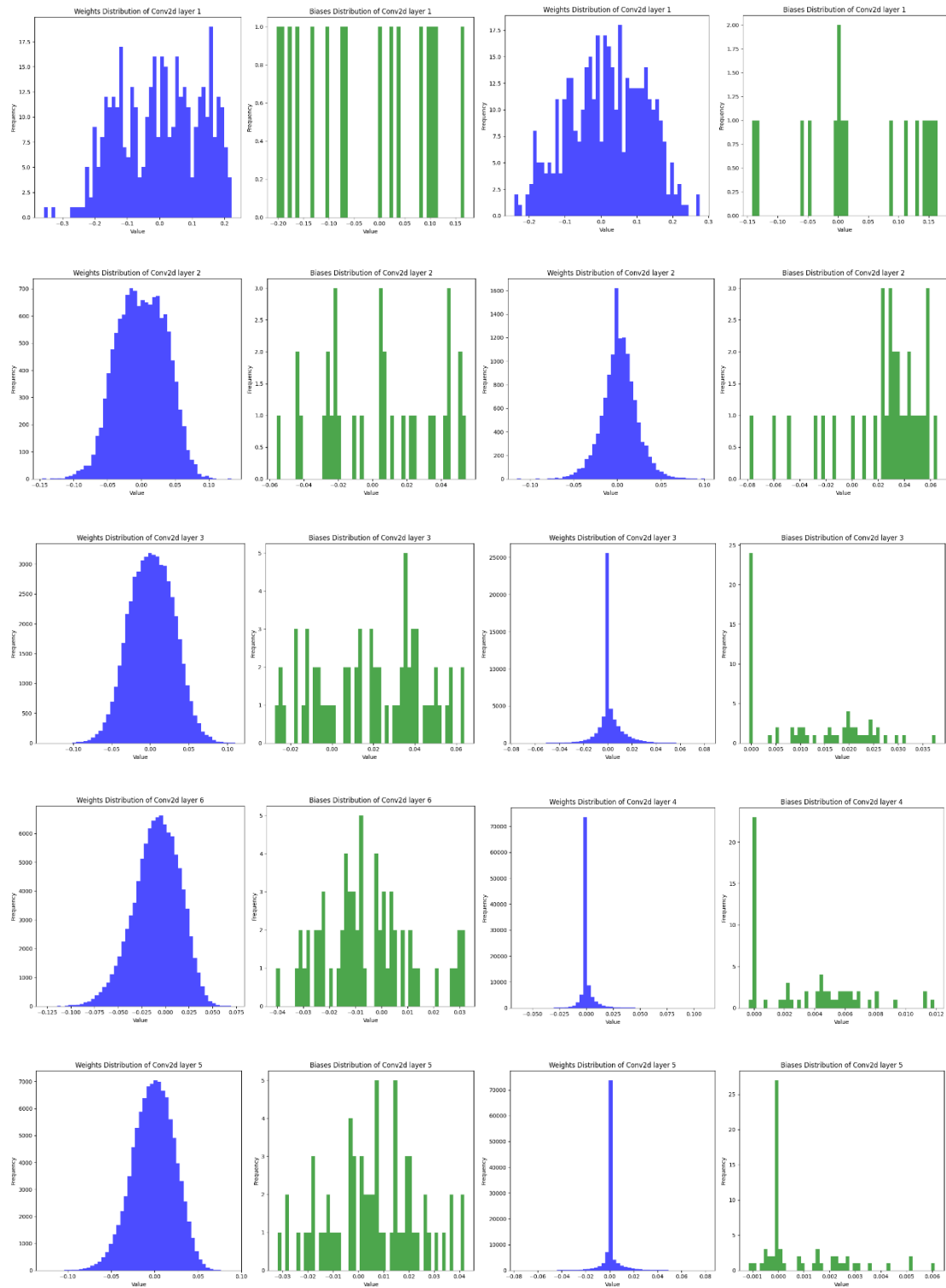
L2 regularization

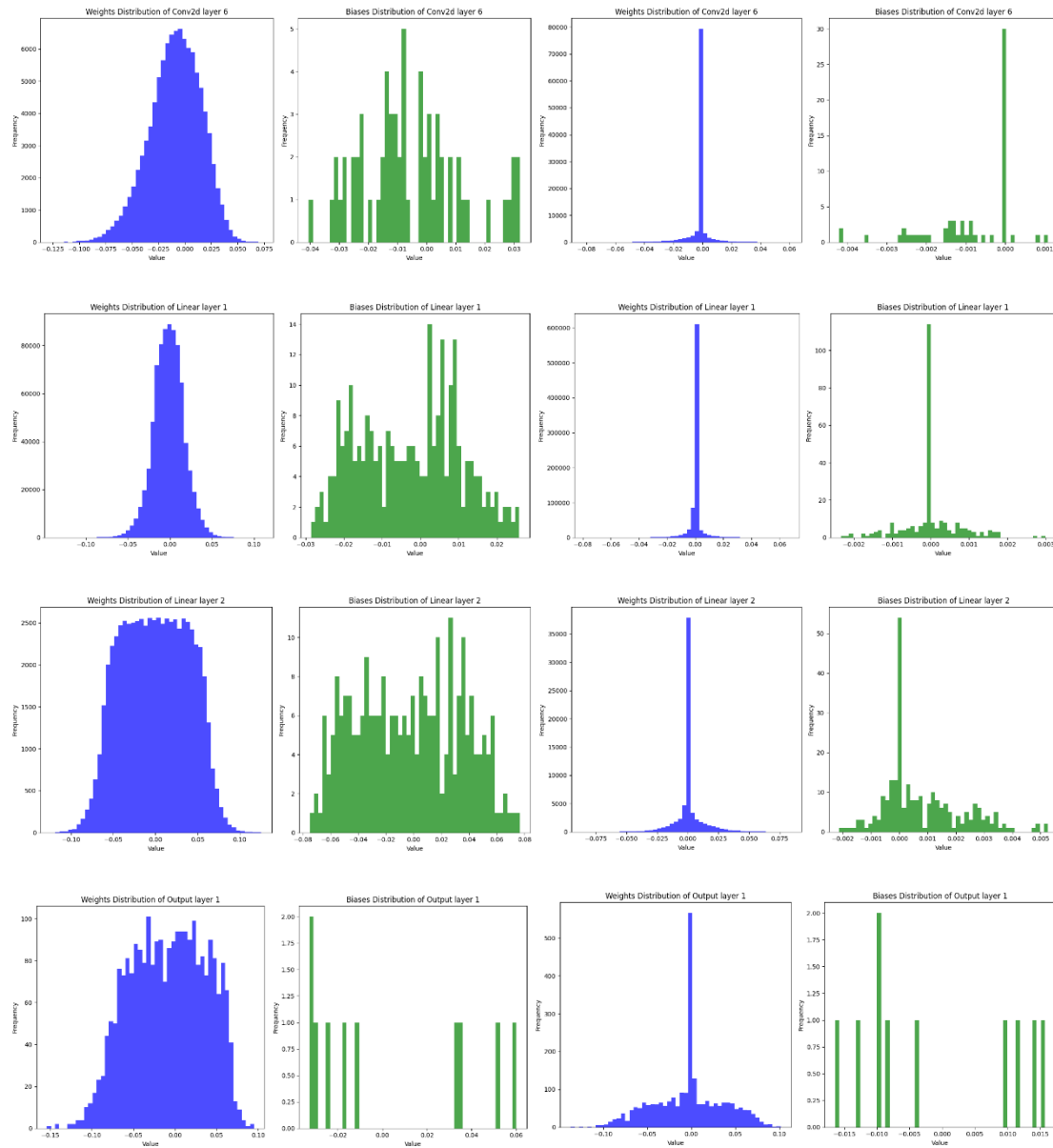


distributions of weights (blue) and biases (green)

No L2 regularization

L2 regularization

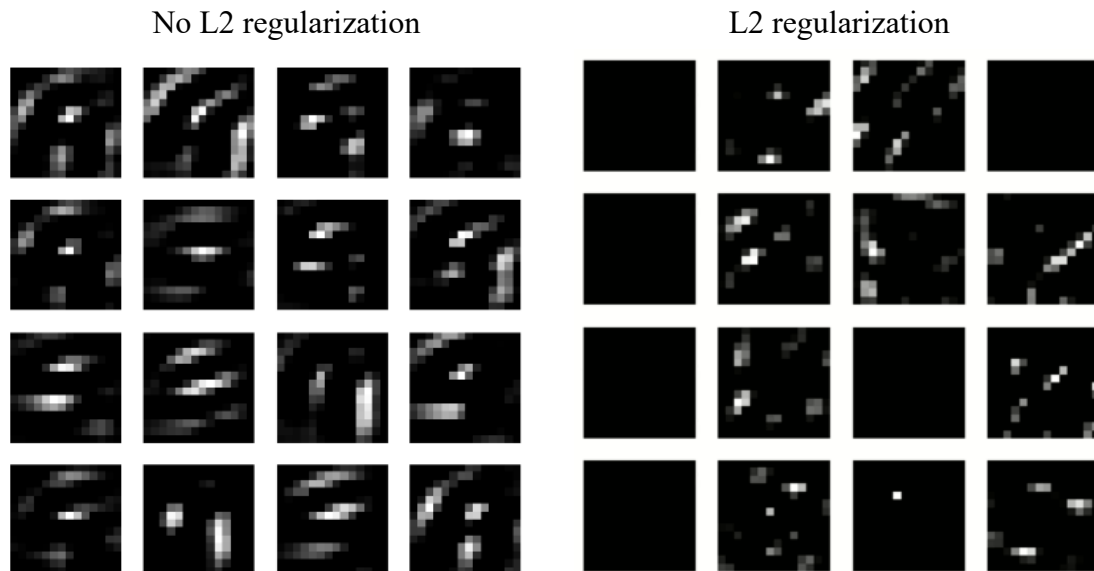




According to the line charts above, this confirms the earlier illustration of L2 regularization. It effectively reduced the risk of overfitting, as shown by the learning curve. Without L2 regularization, the validation and testing losses increased significantly throughout the training process. However, when L2 regularization was applied, the validation and testing losses remained stable at around 0.05, instead of increasing as observed without regularization.

On the other hand, the bar charts also confirm the earlier point that L2 regularization causes the model's weights to approach zero, as indicated by the long

bar at zero. This demonstrates how L2 regularization helps constrain the magnitude of the weights, contributing to a simpler model that generalizes better.



From the images above, it is evident that L2 regularization has indeed reduced most of the weights close to zero.

2. Preprocessing Before Using Convolutional Neural Network for Image Recognition

2-1

I used a total of 6 convolutional layers with the ReLU activation function and applied max-pooling after the 1st and 6th convolutional layers to reduce the image size from 32x32 to 8x8, halving the size each time. I also added residual links between conv3 and conv4, and between conv5 and conv6. This helps to avoid the problem of gradient vanishing. After these layers, the image passes through 3 linear layers, and the final output consists of 10 neurons representing the 10 different classes of images.

```
class Net(nn.Module):
    def __init__(self, num_classes=10):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 48, kernel_size=5, stride=1, padding=2)
        self.maxpool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(48, 128, kernel_size=5, stride=1, padding=2)
        self.conv3 = nn.Conv2d(128, 192, kernel_size=5, stride=1, padding=2)
        self.conv4 = nn.Conv2d(192, 192, kernel_size=5, stride=1, padding=2)
        self.conv5 = nn.Conv2d(192, 192, kernel_size=5, stride=1, padding=2)
        self.conv6 = nn.Conv2d(192, 192, kernel_size=5, stride=1, padding=2)
        self.fc1 = nn.Linear(192 * 8 * 8, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, num_classes)

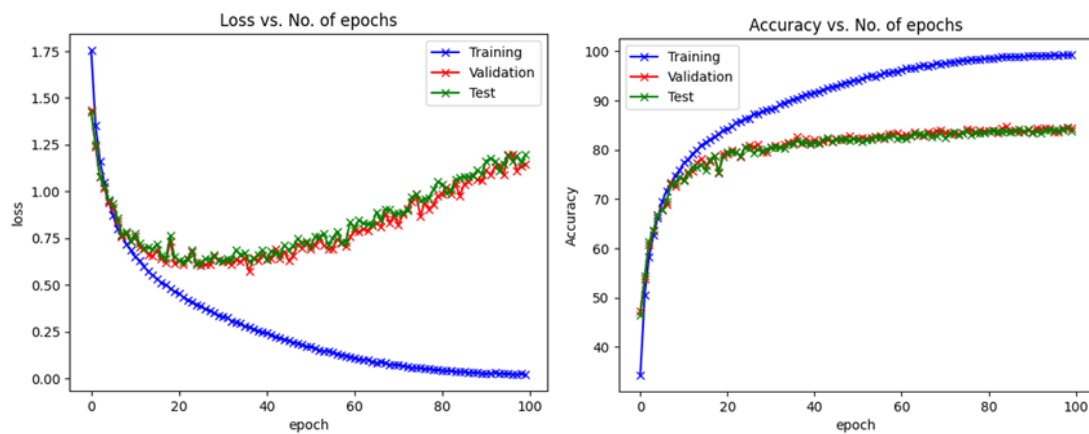
    def forward(self, x):
        x = F.relu(self.conv1(x))
        conv1_out = x
        x = self.maxpool(x)
        x = F.relu(self.conv2(x))
        conv2_out = x
        x = F.relu(self.conv3(x))
        conv3_out = x
        x = F.relu(self.conv4(x) + x)
        conv4_out = x
        x = F.relu(self.conv5(x))
        conv5_out = x
        x = F.relu(self.conv6(x) + x)
        conv6_out = x
        x = self.maxpool(x)
        x = torch.flatten(x, start_dim=1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x, [conv1_out, conv2_out, conv3_out, conv4_out, conv5_out, conv6_out]
```

For my model, the stride = 1 and the filter size = 5

Since the input image is 32*32 pixels, I think a filter size of 5 also perfectly matches the size of image's important features, such as a dog's eyes or the shape of a ears. To avoid losing any features in the image, I decided to use a stride of 1.

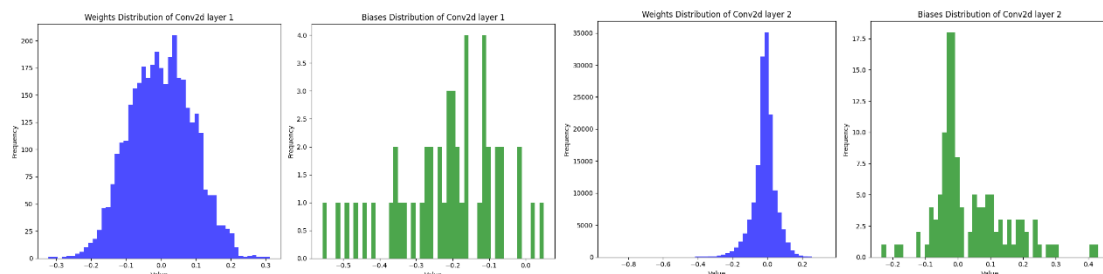
To ensure that the output feature maps have the same size as the input feature maps, I used a padding of 2. This padding ensures that the size of the output feature maps does not decrease continuously, according to the formula mentioned above.

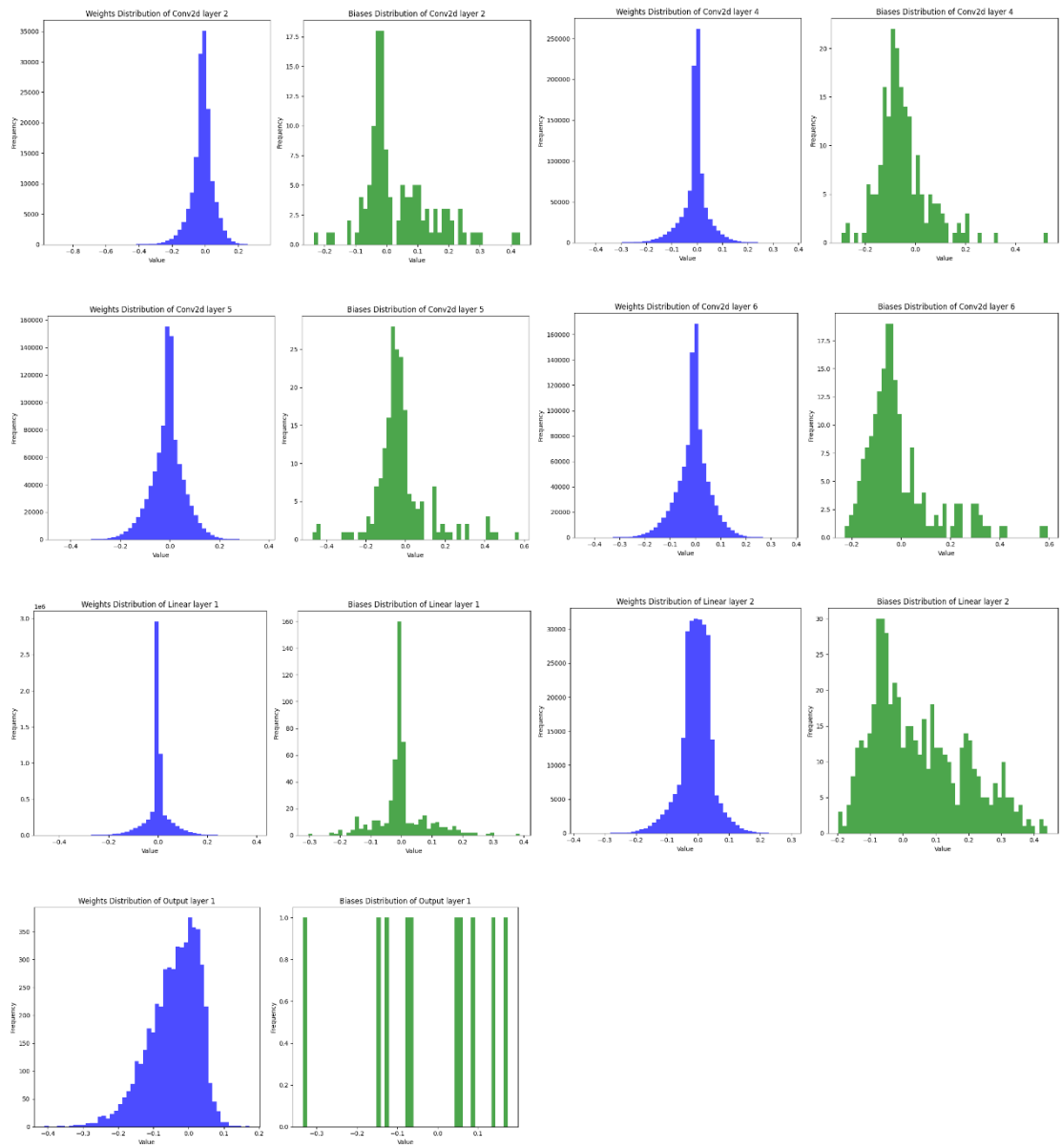
learning curve and accuracy of training, validation and test sets



distributions of weights (blue) and biases (green)

for 6 convolution layers and 3 linear layers



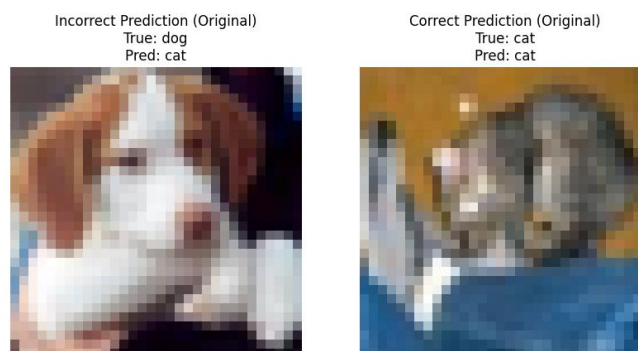


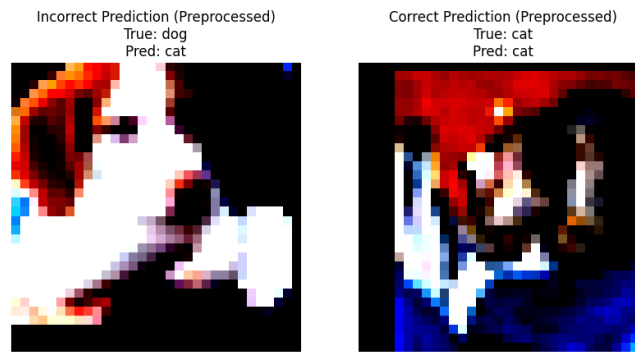
2-2



The example I chose is the horse and the cat. On the left-hand side, it shows the model recognizing it as a cat instead of a horse, and on the right-hand side is an image correctly recognized as a cat.

After preprocessing, the horse's texture disappeared. I think this led the model to misinterpret the two black dots on the horse as a cat's eyes and the shape of the horse as part of a cat's facial coloring.





Another example I chose is the dog and the cat. On the left-hand side, it shows the model recognizing it as a cat instead of a dog, and on the right-hand side is an image correctly recognized as a cat.

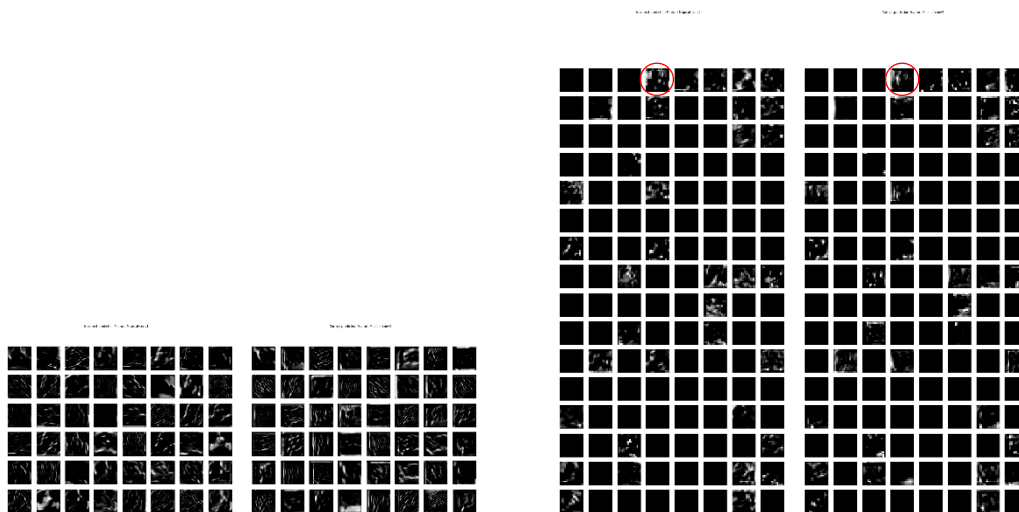
In this case, I think the model's mistake is simply due to the similarity between dogs and cats. Both are cute animals and share similar features, such as eyes and color patches.

2-3

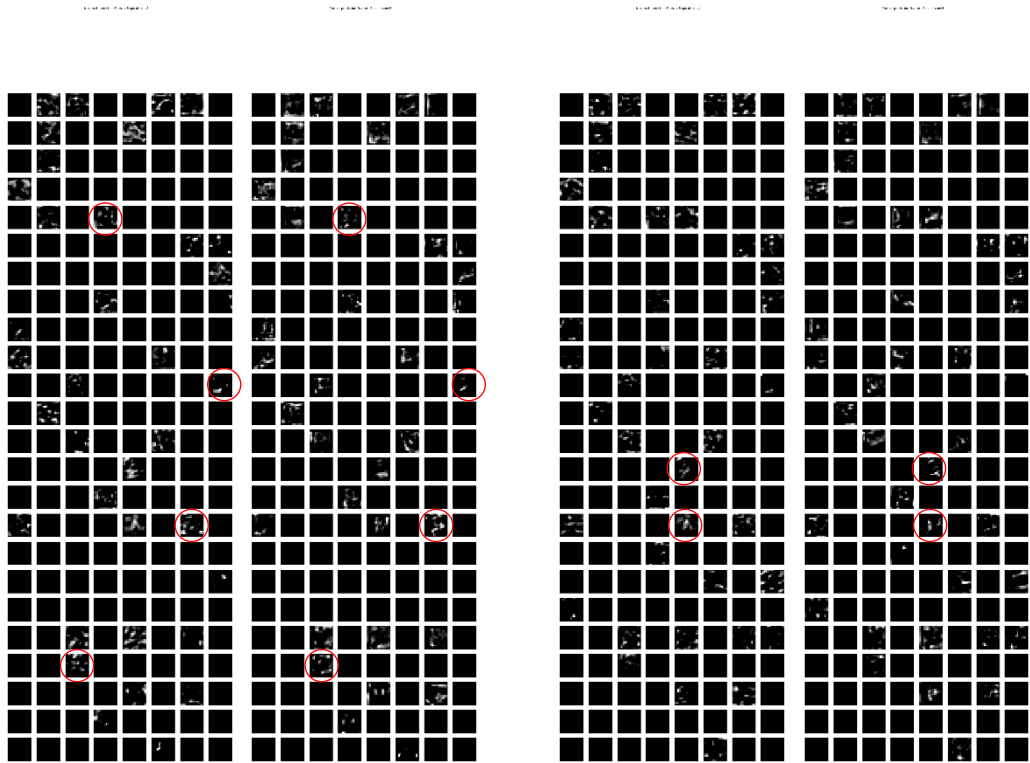
Here are the feature maps output from each convolutional layer. Since the output channels of each layer are 48, 128, 192, 192, 192, and 192, the number of images for each layer's outputs will match the number of its output channels.

Left : incorrect prediction / Right : correct prediction

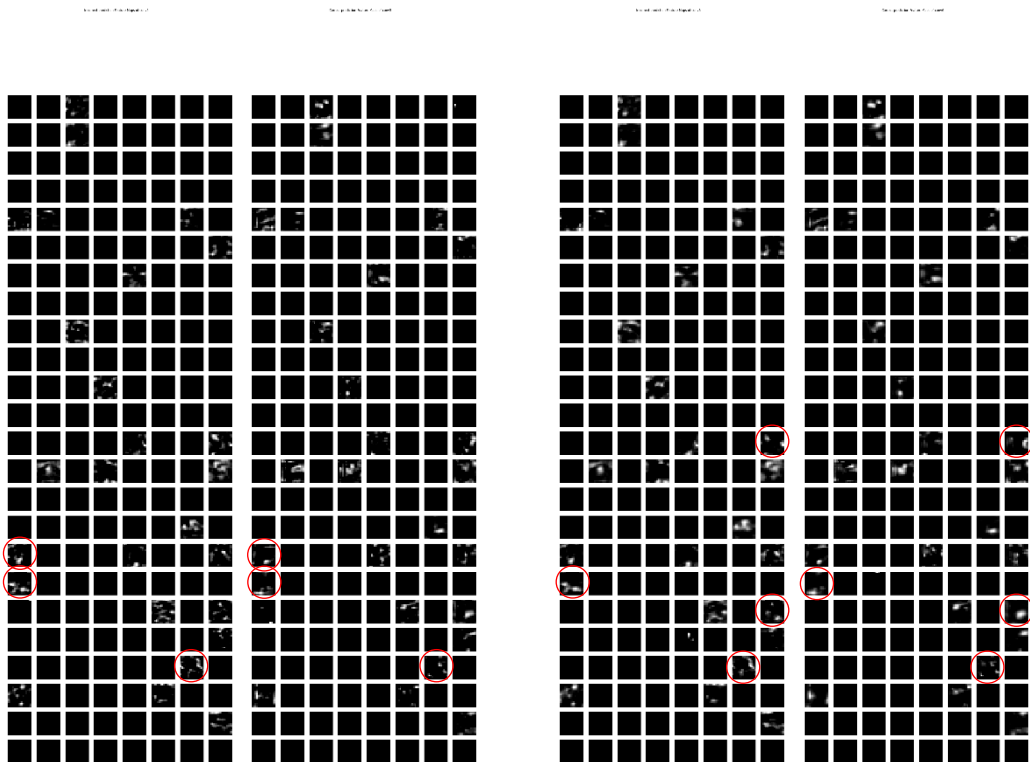
Convolution layer 1 (output channel : 48) Convolution layer 2 (output channel : 128)



Convolution layer 3 (output channel : 192) Convolution layer 4 (output channel : 192)



Convolution layer 5 (output channel : 192) Convolution layer 6 (output channel : 192)



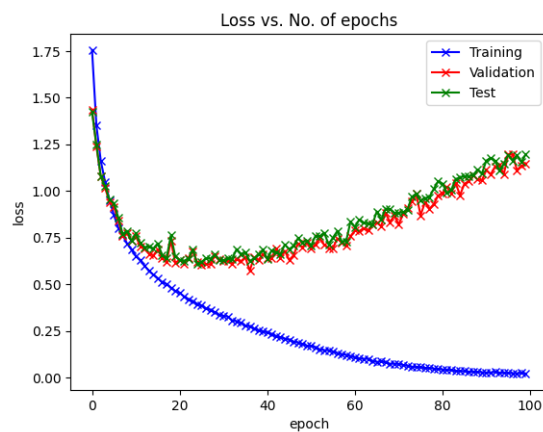
Comparing the two feature maps above, I noticed that there are many similar feature maps in the same positions. This likely caused the model to struggle in distinguishing between the two different classes.

2-4

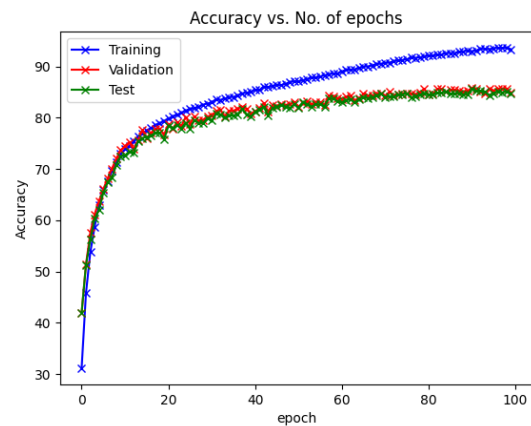
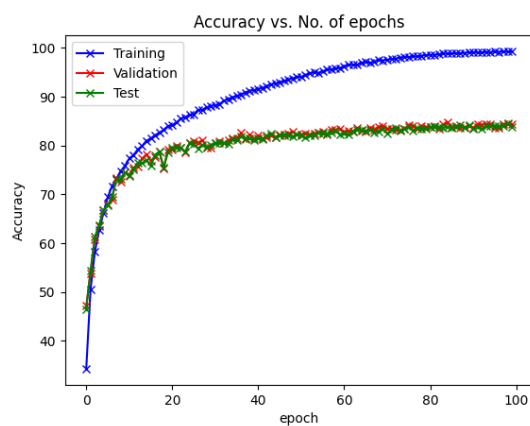
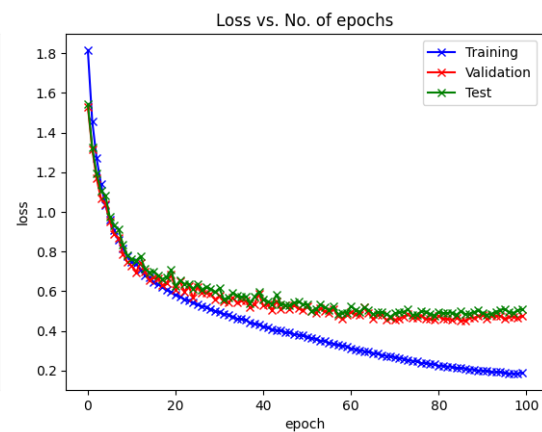
Following 2-1, please add L2 regularization to the CNN implemented in 2-1 and discuss its effect.

Loss and Accuracy

No L2 regularization



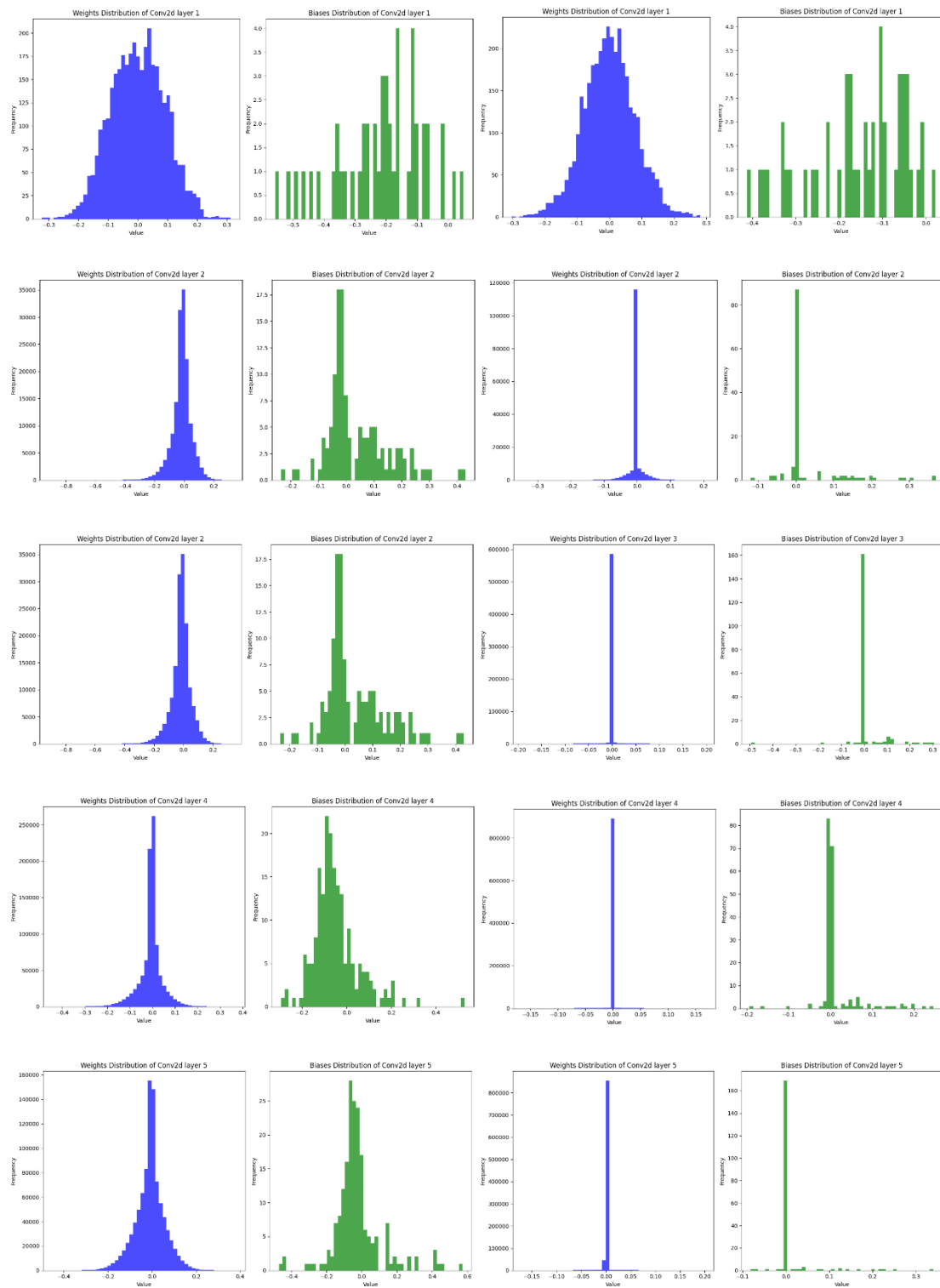
L2 regularization

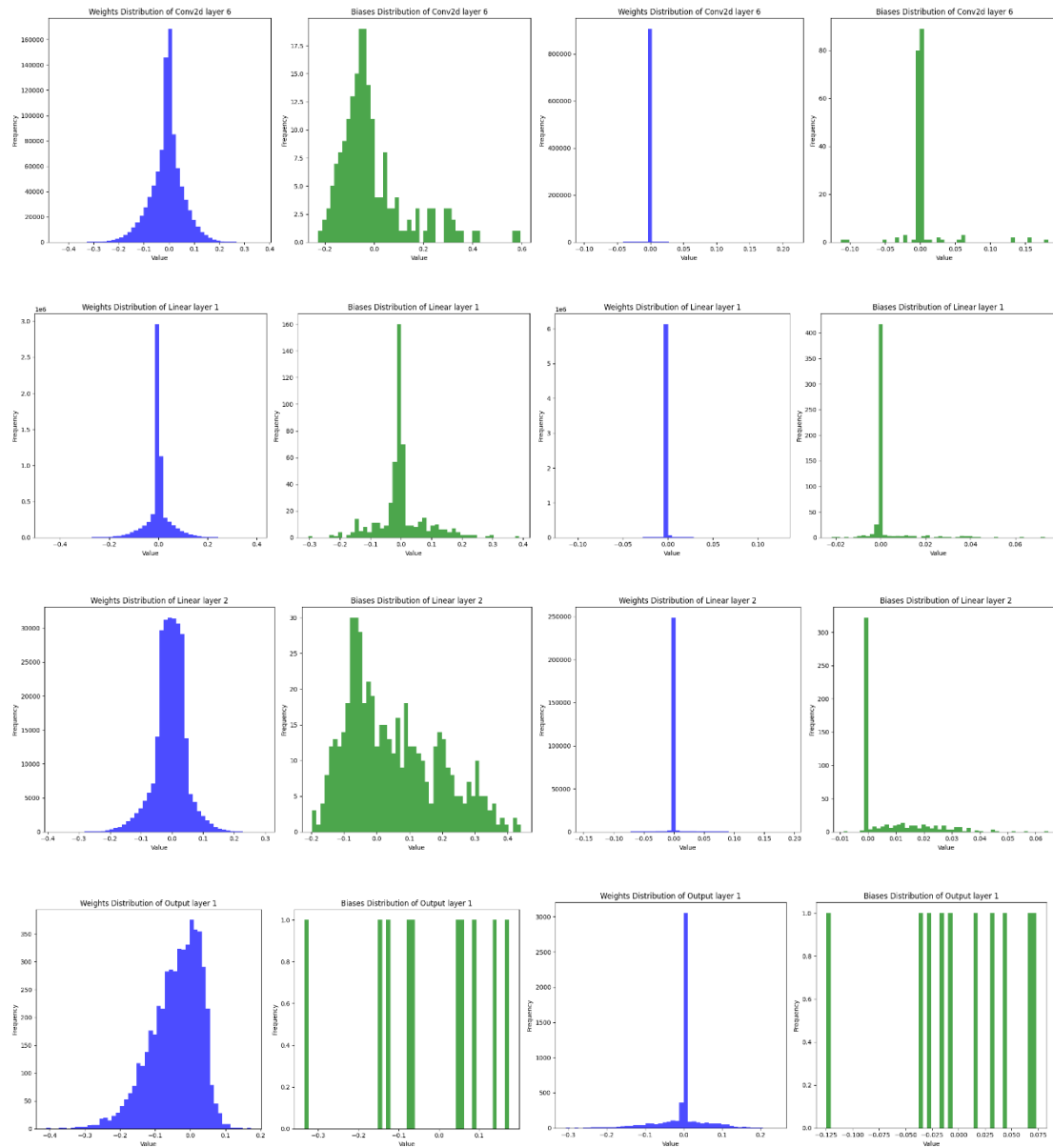


distributions of weights (blue) and biases (green)

No L2 regularization

L2 regularization





According to the line charts above, this confirms the earlier illustration of L2 regularization. It effectively reduced the risk of overfitting, as shown by the learning curve. The learning curve without L2 regularization shows severe overfitting, the validation and testing losses increased significantly throughout the training process. While the curve with L2 regularization does not exhibit this issue, the validation and testing losses remained stable at around 0.05 - 0.06, instead of increasing as observed without regularization.

On the other hand, the bar charts also confirm the earlier point that L2 regularization causes the model's weights to approach zero, as indicated by the long bar at zero. This demonstrates how L2 regularization helps constrain the magnitude of the weights, contributing to a simpler model that generalizes better.

2-5 describe the preprocessing

```
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(), # romdonly flip the image horizontally
    transforms.RandomCrop(32, padding=4), # randomly crop the image by padding 4 pixels
    transforms.ToTensor(), # convert the image to tensor
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)) # normalize the image
])
```

1. **Random Horizontal Flip** (transforms.RandomHorizontalFlip()):

Randomly flips the image horizontally with a probability of 50%. This augment helping the model perform better by learning from mirrored images.

2. **Random Crop with Padding** (transforms.RandomCrop(32, padding=4)):

Pad the image with 4 pixels on each side, then randomly crop a 32×32 section. This transformation introduces variability in object positioning, helping the model perform better to objects appearing in different locations.

3. **Convert to Tensor** (transforms.ToTensor()):

Converts the images to a tensor. This step also scales pixel values to a range of [0,1][0, 1][0,1].

4. **Normalize** (transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))):

Normalizes the image tensor using channel-wise mean and standard deviation values derived from the CIFAR-10 dataset. The first three values represent the mean for each channel, and the last three represent the

standard deviations, as each color image is composed of three channels: red, green, and blue (RGB).

Centering the data around the mean and scaling by the standard deviation improves training stability and convergence, reducing the impact of extreme values that could disrupt the training process.

Example:

Original



Preprocessed

